

Build Plan AEL pipeline agent

Team	Alberto Barrionuevo · Pantelis Paraskevopoulos · Jasper Naalden · Nasjaat Potmis
Build roles this week	Product manager: [Jasper] · Developer: [Naasjaat] · Tester: [Alberto] · Deployer: [Pantelis]
Version / date	v1.0 — [1/10/2026]
Built from	AEL PRD v4 · Technical Blueprint v0.4

1. Phases

Which phase we build first, and why. Phase 1, Appraise. It carries the risk that would hurt most if it turned out to be impossible: our whole design rests on the agent filling the fixed appraised.md contract reliably enough for a person to check rather than redo. If that fails, the file contract fails with it, every later file loses its fields, and no claim in the handbook can be traced to a source. Collect, Translate, Draft, Check and Publish all assume that file already exists, so testing them first would prove nothing. We can build Appraise before Collect exists because a person writes the source.md by hand, which our PRD allows through the manual-fallback constraint.

Phase	What a person can do at the end that they could not do before	What it needs from the phase before
1 · Appraise	Take a source.md and get back a complete appraised.md with every contract field filled, or a clear rejection with a reason.	Nothing. This is the first phase; the source.md is written by hand.
2 · Collect	Give the agent a document, a URL or a sub-question and get a source.md with an id and its origin, without writing the file by hand.	A working Appraise, so a collected source can be appraised immediately and we see whether it was collected well.
3 · Translate + Gateway	Translate an appraised source into English, and see an interview transcript refused until a person has redacted it.	Appraised files from phases 1 and 2 to translate.
4 · Draft	Select source_ids and get a draft chapter following the handbook template, where every claim carries the id of its source.	At least three translated.md files from phase 3.
5 · Check	Send a draft through the three check layers and see a page that fails the criteria returned to Draft instead of published.	A draft.md from phase 4.
6 · Publish	Publish an approved page to the existing site and read every rule the agent follows from one shared context file.	An approved page.md from phase 5.

2. Subtasks inside each phase

One change each, small enough to test. Every subtask carries one build role and one name, and names what it touches.

2.1 Phase 1 Appraise

	Subtask	What it touches (file · blueprint part · prompt · folder)	Role
☐	1.1 Create the repository folder structure	/sources, /appraised, /translated, /drafts, /pages, /instructions — repo root	Developer
☐	1.2 Write the Appraise prompt: fill the nine appraised.md fields; write "not stated in source" rather than guess	/instructions/appraise.md — part Appraise — prompt	Developer
☐	1.3 Add the rule checks: age ≤ 4 years, accepted source_type, language NL or EN	/instructions/appraise.md — part Appraise — PRD 2.4	Developer
☐	1.4 Create the publisher allow-list the rule checks read	/instructions/allow-list.md — part Appraise	Product manager
☐	1.5 Write one source.md by hand from the CBS micro-enterprises report	/sources/cbs-microbedrijven-2026.md	Tester
☐	1.6 Write one source.md by hand from a source older than four years, for the rejection test	/sources/old-source.md	Tester

2.2 Phase 2 Collect

	Subtask	What it touches (file · blueprint part · prompt · folder)	Role
☐	2.1 Write the Collect prompt: assign source_id, record origin, supplier and date, write source.md	/instructions/collect.md — part Collect — prompt	Developer
☐	2.2 Label any transcript as source_type = interview, review_status = pending	/instructions/collect.md — part Collect — PRD 2.4	Developer
☐	2.3 Store the two AIBS 1.5 query templates (EN and NL) as editable instructions	/instructions/queries.md — part Collect	Product manager
☐	2.4 Run Collect on one URL and one uploaded PDF	/sources — part Collect	Tester
☐	2.5 Run Collect from one sub-question using a stored query template	/sources — part Collect	Tester

2.3 Phase 3 Translate + Gateway

	Subtask	What it touches (file · blueprint part · prompt · folder)	Role
☐	3.1 Write the Translate prompt: English output, Dutch terms in brackets, file always written	/instructions/translate.md — part Translate — prompt	Developer
☐	3.2 Write the Gateway rule: refuse any interview whose review_status is not "redacted"	/instructions/gateway.md — Decision 5 / blueprint 1.4	Developer
☐	3.3 Write the redaction step: how a person removes identifying detail and sets review_status	/instructions/redaction.md — Decision 5	Product manager
☐	3.4 Create a fake interview transcript so no real participant data is used in a test	/sources/test-interview.md	Tester
☐	3.5 Run the Gateway test and the translation test	/translated — parts Translate and Gateway	Tester

2.4 Phase 4 Draft

	Subtask	What it touches (file · blueprint part · prompt · folder)	Role
☐	4.1 Store the chapter template (six fixed sections)	/instructions/chapter-template.md — part Draft	Product manager
☐	4.2 Write the Draft prompt: open only the selected source_ids, tag every claim with its source_id	/instructions/draft.md — part Draft — prompt	Developer
☐	4.3 Add the rule that Draft reports a section it has no source for instead of writing an unsupported claim	/instructions/draft.md — part Draft	Developer

☐	4.4 Add APA 7 reference generation from the appraised.md fields	/instructions/draft.md — part Draft	Developer
☐	4.5 Run Draft on three source_ids, and once on a chapter section no source covers	/drafts — part Draft	Tester

2.5 Phase 5 Check

	Subtask	What it touches (file · blueprint part · prompt · folder)	Role
☐	5.1 Check layer 1 (code): source_id on every claim, six sections, APA 7, no identifying detail	/instructions/check.md — part Check	Developer
☐	5.2 Write Check layer 2 (model): flag wording against the quality criteria and ISO 20607 4.4	/instructions/check.md — part Check — prompt	Developer
☐	5.3 Write Check layer 3 (person): who approves, and how a rejection returns to Draft with the failed criterion named	/instructions/check.md — Decision 6	Product manager
☐	5.4 Run Check on a clean draft and on a deliberately broken one	/pages — part Check	Tester
☐	5.5 Run Check on a draft that names an interviewee	/pages — part Check	Tester

2.6 Phase 6 Publish

	Subtask	What it touches (file · blueprint part · prompt · folder)	Role
☐	6.1 Write the Publish prompt: publish to the existing site and record the URL on the sources	/instructions/publish.md — part Publish — prompt	Deployer
☐	6.2 Consolidate all rules, criteria and terminology into one shared context file	/instructions/shared-context.md — Decision 4	Developer
☐	6.3 Write the manual fallback checklist: how a person produces each of the five files by hand	/instructions/manual-fallback.md — all parts	Product manager
☐	6.4 Publish one real page and record its URL	/pages, site — part Publish	Deployer

3. Test gates

One gate per phase: what a person runs, and the result that counts as passing. The tester runs the three steps and writes what was seen in section 6.

3.1 Gate 1 · Appraise

1. Run the Appraise prompt on /sources/cbs-microbedrijven-2026.md.
2. Open the output file and compare it field by field with the contract in blueprint 1.3; pick three key_findings at random and search for them in the source text.
3. Run the Appraise prompt on /sources/old-source.md.
Passes when: step 1 gives all nine fields with the exact names, publication_date matches the source, the three key_findings are in the source text; step 3 is rejected with a reason and no file is written.
PRD quality this gate keeps: PRD 2.3.3 traceable to sources · PRD 2.4 sourcing rules.

3.2 Gate 2 · Collect

1. Run Collect on one URL and on one uploaded PDF.
2. Run Appraise on both results without editing the files by hand.
3. Run Collect for sub-question 4 using the stored query template in /instructions/queries.md.
Passes when: each source gets a unique source_id with its origin, Appraise accepts both unedited, and step 3 returns at least three candidates, each accepted or rejected with a reason.
PRD quality this gate keeps: PRD 2.2.1 collect sources · PRD 2.4 sourcing rules and agentic-CLI operation.

3.3 Gate 3 · Translate + Gateway

1. Ask the agent to translate /sources/test-interview.md while review_status is "pending".
2. Set review_status to "redacted" by hand and ask again.
3. Run Translate on one Dutch source and on one English source.
Passes when: step 1 is refused with the Gateway rule named and nothing of the transcript leaves; step 2 is processed normally; step 3 gives English with MKB and two other Dutch terms in brackets, and a file for the English source too.
PRD quality this gate keeps: PRD 2.4 handling of interview material · PRD 2.5 prohibited inputs · PRD 2.3.2 clear English.

3.4 Gate 4 · Draft

1. Select three source_ids and run the Draft prompt.
2. Check the draft against the chapter template and search /appraised for each source_id used.
3. Ask for a chapter section that none of the three sources covers.
Passes when: the draft has the six sections, every claim carries a source_id that exists in /appraised, references are APA 7 from the appraised fields; step 3 reports the missing source instead of writing it.
PRD quality this gate keeps: PRD 2.3.1 consistent structure · PRD 2.3.3 traceability · PRD 2.3.5 APA citations.

3.5 Gate 5 · Check

1. Run Check on the clean draft from phase 4.
2. Delete one source_id from a claim on purpose and run Check again.
3. Run Check on a draft that names an interviewee.
Passes when: step 1 reaches page.md only after a named person approved it; step 2 is rejected with the failed criterion named and returns to Draft; step 3 is rejected for naming an interviewee.
PRD quality this gate keeps: PRD 2.3.1–2.3.5 output qualities · PRD 2.3.6 protect interview participants.

3.6 Gate 6 · Publish

1. Run Publish on the approved page.
2. Open the URL in a browser and check the source records for that URL.
3. Change one rule in /instructions/shared-context.md (for example the recency limit from four years to three) and re-run Appraise on a source published three and a half years ago.
Passes when: the page is reachable at a URL on the existing site, the URL is written back on the sources, and step 3 changes behaviour without editing any role instruction.
PRD quality this gate keeps: PRD 2.4 existing infrastructure, non-coder operation and manual fallback.

4. Commit gate

The rule. Nothing is committed until its tests pass.

What a commit contains. One subtask, one commit. The commit message is the subtask number and what changed, for example: "1.2 Appraise prompt fills the nine contract fields".

What happens to a change whose test fails. It is not committed. It stays on the working copy, the box in section 6 stays unticked, and the failure is written in the log the same day with what the test showed. The subtask returns to the person named on it in section 2, who changes the prompt or the file and runs the same test again. The failed line is never deleted: it is the evidence of our week.

5. Push gate

The rule. The next phase starts only after a successful merge and push. A phase that works on one laptop is not finished.

How work reaches the main branch. Each phase is built on its own branch, named phase-1 to phase-6. When every subtask of the phase is committed and its test gate has passed, the person who built the phase opens a pull request.

Who merges. The deployer, [NAME]. Before merging, the deployer checks that every box of that phase is ticked and that every ticked box has a date, a commit and a line saying what the test showed. A phase with an empty evidence line is not merged.

What a failed merge means for the phase that was waiting. The waiting phase does not start. The deployer and the author resolve the conflict the same day. If it is not resolved, the waiting phase moves to the next session, the delay and its reason go in the log, and the product manager decides whether it may start from the last pushed version.

6. Boxes, and what happened beside them

Who ticks, and when. The tester ticks a subtask box immediately after running its test, in the same session, and writes the evidence line. The deployer ticks the gate box and the merge box after the push. The pen holder checks before every session that the ticks match reality. Nobody ticks a box for work they did not see.

Failures. A failure stays in. If a test failed on Tuesday and passed on Wednesday, both lines are in the table.

	Item	Date	Commit / PR	What the test showed
☐	1.1 Folder structure	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	1.2 Appraise prompt	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	1.3 Rule checks	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	1.4 Allow-list	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	1.5 Hand-written source	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	1.6 Old source for rejection test	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	GATE 1 · Appraise ☐ test passed ☐ merged and pushed	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	2.1 Collect prompt	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	2.2 Interview labelling	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	2.3 Query templates	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	2.4 Run on URL and PDF	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	2.5 Run from sub-question	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	GATE 2 · Collect ☐ test passed ☐ merged and pushed	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	3.1 Translate prompt	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	3.2 Gateway rule	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	3.3 Redaction step	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	3.4 Fake transcript	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	3.5 Gateway and translation runs	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	GATE 3 · Translate + Gateway ☐ test passed ☐ merged and pushed	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	4.1 Chapter template	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	4.2 Draft prompt	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	4.3 Missing-source rule	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	4.4 APA references	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	4.5 Draft runs	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	GATE 4 · Draft ☐ test passed ☐ merged and pushed	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	5.1 Check layer 1	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	5.2 Check layer 2	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	5.3 Check layer 3	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	5.4 Clean and broken draft	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	5.5 Interviewee test	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	GATE 5 · Check ☐ test passed ☐ merged and pushed	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	6.1 Publish prompt	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	6.2 Shared context file	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	6.3 Manual fallback checklist	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	6.4 Page published	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]
☐	GATE 6 · Publish ☐ test passed ☐ merged and pushed	[DATE]	[COMMIT / PR]	[WHAT THE TEST SHOWED]

7. The rules, written out

The three gates as instructions someone new could follow without asking.

7.1 What to run (test gate)

1. Find your subtask in section 2 and its phase gate in section 3.
2. Run the three steps of the gate exactly as written, on the files named there. Do not use a different example.
3. Write one line in the table in section 6: the date, the commit, and what you saw. Write what happened, not whether you are satisfied.
4. A test is a person doing those three steps and recording the result. An agent reporting "done" is not a test.

7.2 When to commit

1. Only after the test of that subtask has passed and its line is written in section 6.
2. Commit one subtask at a time. Message: subtask number plus what changed.
3. If the test did not pass, do not commit. Leave the box unticked, write the failure line, and give the subtask back to the person named on it.

7.3 When to push

1. When every subtask of the phase is committed and the phase gate has passed, open a pull request from the phase branch.
2. The deployer checks the boxes and the evidence lines, then merges and pushes to the main branch.
3. Only then does the next phase start. Tell the team in the group chat that the phase is pushed.

7.4 What happens when a gate does not open, and after how many tries

1. First try fails: the person named on the subtask changes the prompt or the file and runs the same test again, the same day.
2. Second try fails: stop, never a third try in silence. The product manager either changes the subtask, changes the test, or carries it to next week as an open question with an owner.
3. Whatever is decided is written in section 6 beside the unticked box, with the date and the reason.
4. If a phase gate does not open after two attempts, the phase is not merged and the next does not start; the product manager reduces the phase or moves it.
5. If a merge fails twice, the deployer and the author fix it together the same day; if it still fails, the last pushed version stays and the conflict goes in section 6.